

Animated Visual Encoding and Layer Blending for Identification of Educational Game Strategies

Braden Roper^{*}
K20 Center
The University of Oklahoma

William Thompson[†]
K20 Center
The University of Oklahoma

Chris Weaver[‡]
School of Computer Science
The University of Oklahoma

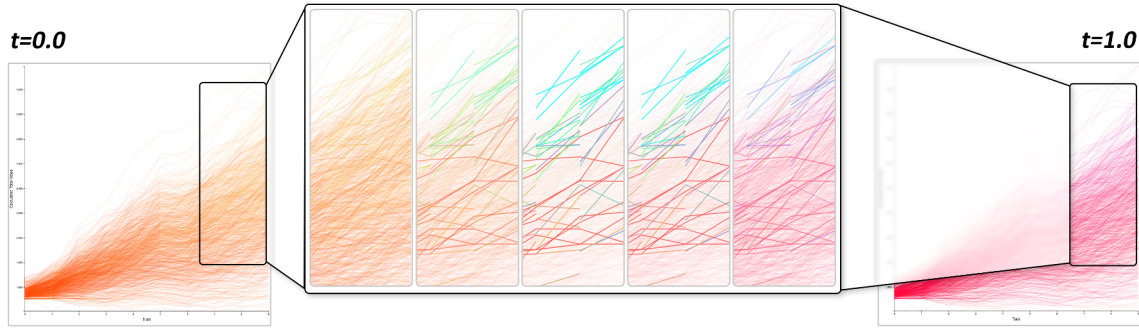


Figure 1: An example of animated output of a kinetic query, revealing a successful strategy in the election-simulation and computational thinking educational game Operation: ELECT. The line plot shows calculated acquired votes over the turns of an individual level. Isolated success in two key districts of the specific in-game level are encoded with red-to-green and red-to-blue color scales configured to overlap near the midpoint of the animation, creating a cyan blended encoding for players who were successful in both districts. An opacity encoding is also blended in near the midpoint of the animation to accentuate in-game actions of interest.

ABSTRACT

Game-Based Learning has proven to be an effective method for enhancing engagement with educational material. However, gaining a deeper understanding of player strategies remains challenging. Sequential game-state and action-based tracking tools often gather extensive data that can be difficult to interpret as long-term strategy. This data presents unique problems to visualization, as it can be fairly natural, noisy data but is constrained within synthetic, controlled environments, leading to issues such as overplotting which can make interpretation complicated. We propose an animated visual encoding tool that utilizes kinetic visualization to address these issues. This tool enables researchers to construct animated data narratives through the configuration of parameter interpolation curves and blending layers. Finally, we demonstrate the usefulness of the tool while addressing specific interests as outlined by a domain expert collaborator.

Index Terms: Kinetic visualization, kinetic queries, animated encoding, game-based learning

1 INTRODUCTION

In recent years, a need for computational thinking and data literacy education has been identified, with more jobs requiring familiarity with data and advanced software systems, even outside the field of Computer Science [5]. Educational standards [6] and models [9] are new and still being refined. The K20 Center's educational game Operation: ELECT [10] was developed to aid in this new computational thinking education, while also connecting to social studies

standards for electoral processes in the United States. In the game, students play as campaign managers who work to get their candidate elected. They are presented with simulated polling data, news events, and regional interests that they interpret to inform their decisions. To be successful, a player cannot simply address the specific circumstances of a given turn, but must also develop a long-term strategy. The game's primary Instructional Designer—and second author of this paper—is especially interested in analyzing and visualizing player strategy to evaluate the balance of game mechanics and to confirm the game's efficacy as an educational resource.

In this paper, we explore the use of animated visualization as a means to reveal patterns in complex gameplay data. If a simple line chart can represent per-player progression of a single state variable, then it follows that additional visual encoding can be applied to per-turn line segments to depict other analytically relevant state variables. The main building blocks of all graphical elements in visualization systems, known as visual encodings, typically include attributes such as position, size, shape, color, and orientation. For this application we focus solely on color and opacity due to their flexibility and blending properties when encoding information onto an underlying line plot. The proposed technique extends these channels through animation to offer a new method to analyze complex game data. This is done through the introduction of *kinetic queries*, which can be thought of as an extension of dynamic queries [2, 3]. In addition to specifying the data parameters and the encoding channels that will be used, kinetic queries include an animation curve by which the encoding will be applied and the blending mode used to accumulate the overall visual mapping. This gives analysts the flexibility to stack or sequence visual effects.

To demonstrate the usefulness of the proposed tool, we perform example analyses on gameplay data. Key interests are identified by our domain expert collaborator and aligned to the three provided examples. Although the given analyses focus solely on encoding information into the color and opacity of animated line charts, our ultimate goal is to further develop this technique and include other encoding channels and types of visualizations.

^{*}e-mail: bradenroper@ou.edu

[†]e-mail: will.thompson@ou.edu

[‡]e-mail: cweaver@ou.edu

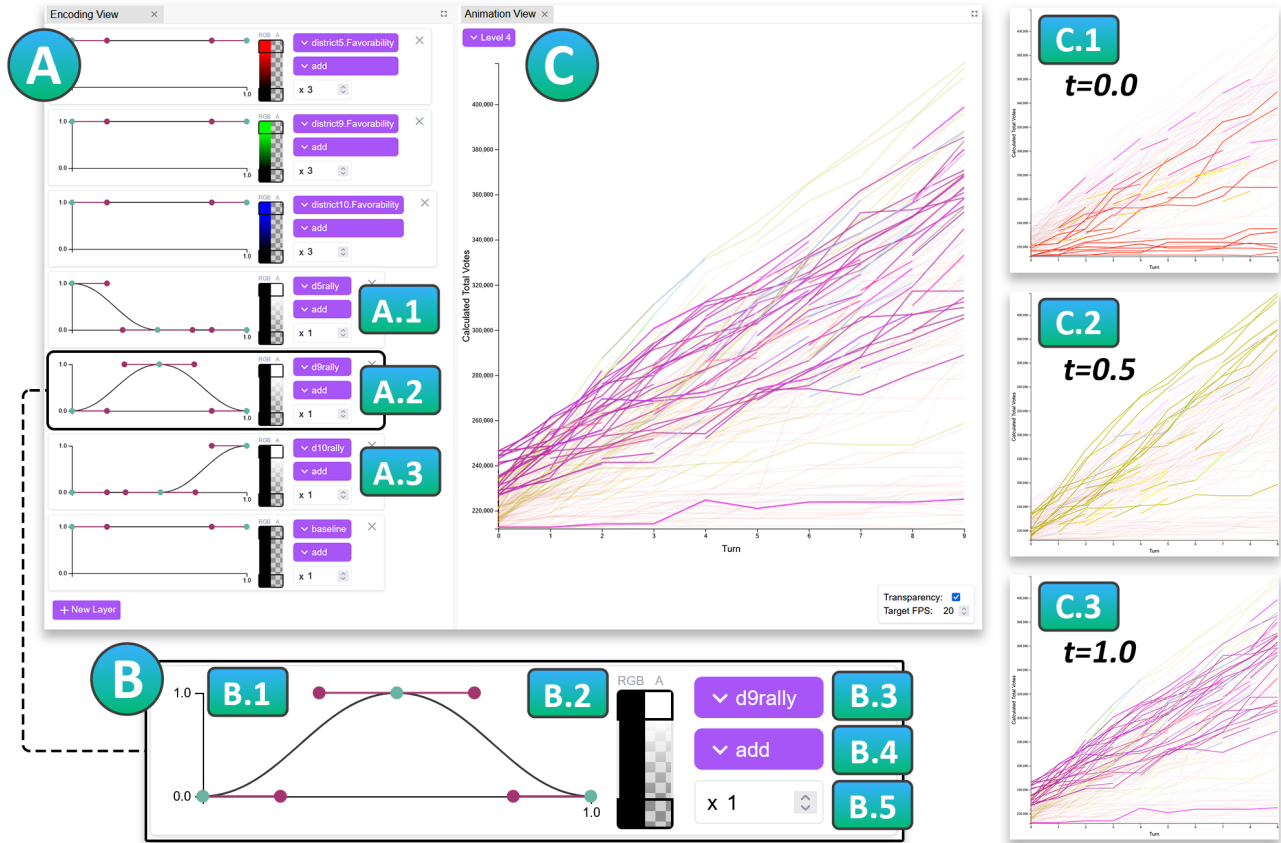


Figure 2: The full interface of the tool has two main views: the Encoding View and the Animation View. The Encoding View (A) allows analysts to add and configure layers, one of which (B) is expanded here to show its inputs (B.1-5) in greater detail. The Animation View (C) continuously plays an animated visual based on the configured layers. The full effect of the technique cannot be seen here due to its animated nature. However, multiple time-steps (C.1-3) can be seen at times $t = 0.0$, $t = 0.5$, and $t = 1.0$, where the animation curves in A.1-3 are at their peaks.

2 RELATED WORK

Game-Based Learning activities have been prized for their ability to increase engagement, adapt to the circumstances of a specific player, and allow graceful failure [8, 15]. Successful implementation of these games requires careful instructional design, which often includes proper incorporation of educational material, a balance of player cognitive load, and the right amount of guidance [19, 20]. Much of these design skills and templates are powered by analyzing game data that, in recent years, has greatly increased in volume and complexity, prompting research in the evolving field of Learning Analytics [7, 16].

Although there are many examples of visualization for game data [13, 14, 18, 23], representing complex game states and their changes over time remains difficult. Such difficulties have led us to explore animation techniques such as kinetic visualization, designed to utilize our perceptual abilities to detect patterns in motion [11, 12].

Early implementations of kinetic visualization showed promise, such as the use of *moxel* displays to explore geospatial data [4, 24]. However, further research in the field is surprisingly sparse, and work that does exist often limits the use of the time dimension for the exclusive representation of the temporal aspect of a dataset [17, 21]. While this use of time is conceptually natural, we wish to look past this restriction and encourage additional exploration into more flexible uses of animation.

3 APPROACH

The proposed technique aims to create animated visualizations by adding kinetic queries in the form of layers. The tool is divided into two main interfaces: the Encoding View (Fig. 2.A) and the Animation View (Fig. 2.C).

3.1 Encoding Layers

The Encoding View (Fig. 2.A) allows analysts to add any number of layers, each with a dedicated purpose to link a data parameter to a visual effect. Each layer has a variety of inputs, including an animation curve (Fig. 2.B.1), a color scale (Fig. 2.B.2), a parameter selection dropdown (Fig. 2.B.3), a blending mode selection dropdown (Fig. 2.B.4), and an additional multiplier (Fig. 2.B.5). The color scale previews its color channels and transparency channel separately since an analyst may wish to control these independently, and low transparency values by their nature would make a color selection difficult to interpret. The functional purposes of these inputs, as well as the methods used to combine them, are described in more detail in Sec. 3.2.

Each layer will output a color object that contains three color channels and a transparency, or alpha, channel. At a given time step, every layer is processed at the same time t with layers being processed sequentially starting from the top and passing their outputs to the layer below them. The final layer dictates the visual encoding to apply to a data point in the Animation View (Fig. 2.C). This means that by scanning vertically across the layers, an analyst can

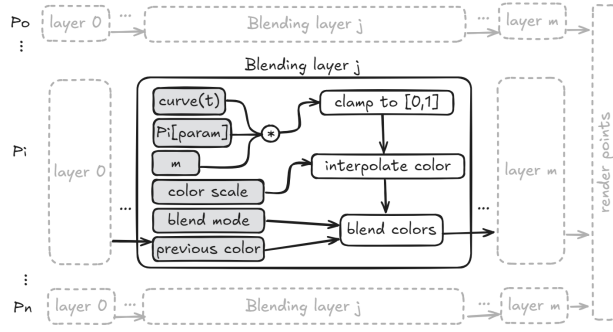


Figure 3: A layer processing diagram is shown. At any time t , each point P_i has a calculated visual encoding determined by its sequential flow through every blending layer. The input elements are shaded, with the first five being set through the blending layer's interface. The final input is provided as output from the previous layer.

stack the desired visual effects by modifying each layer's animation curve, allowing their kinetic queries to be applied sequentially, in sync, or with any desired amount of overlap.

3.2 Layer Processing and Blending

At each time step, each visualized data point will be processed through all defined encoding layers. Each layer has the inputs outlined in Sec. 3.1, with an additional input color provided as output from the previous layer, as seen in Fig. 3. In the case of the first layer, a constant color is assumed for the previous color, with 0 values for all color and transparency channels.

To process a data point in a single layer, we must first obtain the interpolation parameter c , found by the following equation:

$$c = A(t) * p * m$$

where $A(t)$ is the animation curve's value at time t , p is the value of the chosen parameter for the current data point (or alternatively a constant *baseline* option), and m is the additional multiplier, which is 1 by default. This product c is then clamped to the range of $[0, 1]$ and used as the interpolation parameter along the specified color scale (Fig. 2.B.2), resulting in a color value, including transparency. The specified blending function (Fig. 2.B.4) then combines this color with the result from the previous layer, returning a new color that is passed along to the next layer in sequence or, in the case of the final layer, to the Animation View for rendering.

The blending functions used in this work are inspired by those of image and video editing software [22]. We include three basic blending functions. *Add* and *Multiply* apply simple arithmetic operators at the level of individual color and alpha channels. *Mask* takes the minimum alpha value from the two inputs and ignores the main color channels, copying them from the previous layer.

3.3 Animated Output

The Animation View (Fig. 2.C) constantly loops over a visualization based on the layers specified in the Encoding View (Fig. 2.A). For the scope of our provided examples and analyses in Sec. 4, we use kinetic encoding to enhance line graphs. Adding new encoding layers, or kinetic queries, will automatically update the visualization, providing quick feedback to analysts.

The additional frames seen in Fig. 2.C.1-3 are not part of the interface but are simply the rendered output at different time steps in which the animation curves of a few layers (Fig. 2.A.1-3) are at their peaks, meaning their contributed visual effects are at an extrema. To observe the animated output of this technique, refer to the provided supplemental video.

4 ANALYSIS AND FINDINGS

We now demonstrate the use of this technique to explore collected game data from 378 students across Oklahoma. These students attend schools that are partnered with the K20 Center through federal GEAR UP grants. Operation: ELECT uses a tracking system based on the xAPI standard [1], which records user actions, the context in which those actions were taken, and the resulting changes to a player's game state.

For this analysis, we focus on one game level at a time, plotting a player's total votes against their turn and exposing per-turn parameters such as *budget* and *duration*, as well as a set of parameters for each district. On each turn, every district has: a *population*, a candidate *favorability* score, percentages of population that are *unregistered* or *undecided*, percentages of population that are *for* or *against* the player's candidate, and finally a binary label for each type of *action*, with 1 representing that the action was taken in the given district on the given turn and 0 representing that it was not.

4.1 Key Interests

The primary Instructional Designer of Operation: ELECT—and second author of this paper—identified a few Key Interests (KIs) when considering strategic analysis of players, efficacy of the game as an educational activity, and balance of the game's mechanics.

KI.1 While assumptions can be made about the use of displayed district data in player decision-making, it is difficult to interpret long-term strategy spanning multiple turns of the game. Identification of strategy is especially crucial for Operation: ELECT, as it corresponds to the algorithmic thinking that is core to educational computational thinking standards.

KI.2 We are interested in comparing the use of individual actions and highlighting any common combinations of actions. It is also of interest to identify any correlation between individual or combinations of actions and when they are taken in a playthrough.

KI.3 In general, we would like to identify any areas of imbalance in the game. While it is intended for players to identify useful and successful strategies, imbalanced game mechanics may hinder learning objectives.

KI.4 We are interested in districts that have high, or highly patterned, activity. For instance, frequent action-district pairs may indicate common solutions or mistakes that the designer would like to be aware of.

KI.5 As is common in educational game research, we are interested in the efficacy of the game as a learning tool. For Operation: ELECT, one indication of this efficacy is the convergence of strategic behaviors in players, observed as a gradual reduction in exploratory behavior and an increase in exploitative strategies.

4.2 Examples

Based on the Key Interests outlined above, we analyzed three examples using the proposed technique.

Example 1. Using the tool, successful playthroughs of level one reveal significant activity in districts one and two (KI.4). This makes sense as these districts initially contain higher percentage of population against the player's candidate, meaning player focus in these districts is more fruitful when attempting to achieve the popular vote. In addition, many successful players used fundraiser actions in district four, where players are likely to get more money due to their initially high support.

As seen in Fig. 1, the tool also aids in visualizing these phenomena by encoding red-to-green and red-to-blue color scales on the percentages of population for the player's candidate in these districts. High values from both color scales (green and blue) form the resulting cyan region near the middle of the animation. Masking layers are also added to encode the use of specific actions, utilizing opacity to highlight a particularly successful combination of actions (KI.2). Namely, we encode high opacity on turns where users

exercise grassroots actions in district one and two and a fundraiser action in district four. The resulting visualization reveals a strategy that shows success in districts one and two, which unsurprisingly correlates with higher total votes for the level.

Example 2. In level four, we first noticed two visible clusters of lines. Most move upwards as players gain votes while some remain largely horizontal (see Fig. 2.C, with the bottom cluster most visible in Fig. 2.C.1). We quickly discovered shorter turn durations for this lower cluster, possibly identifying students who hurried through the level and, as a result, made less informed decisions.

Continuing our analysis, we focus on rally actions in districts five, nine, and ten (KI.2). Our Instructional Design expert showed special interest in the use of rallies because they are crucial for increasing *favorability* and are expensive, requiring an *appearance* (another type of in-game currency) in addition to their monetary cost. As explained to players in the game: higher *favorability* scores result in more effective actions and cause more *undecided* voters to swing your way on election day.

We encode red, green, and blue color channels for *favorability* in these districts (see first three layers in Fig. 2.A). We give them flat animation curves so their colors remain constant in our animation, unlike the layers seen in Fig. 2.A.1-3, whose curves animate opacity where rally actions are taken. The resulting animation contains three main phases: (Fig. 2.C.1) district five rallies are shown, with a mostly red-orange hue, (Fig. 2.C.2) district nine rallies are shown, with a mostly yellow hue, and (Fig. 2.C.3) district ten rallies are shown, with a mostly magenta hue. Note that since colors are actually static in this animation, their apparent change is caused by the fading in and out of data points based on the rally actions in different districts. Interestingly, in addition to their own district, the latter two phases show that players have high *favorability* in district five (seen as yellow and magenta, which are formed with a significant red channel contribution), as opposed to the first phase which shows a high *favorability* in district five but not in the other two districts. Additionally, these two phases show generally higher total votes, meaning these players' strategies were more successful (KI.1).

From this visualization, a few conclusions can be drawn. First, spending valuable resources on rallies in district five is less successful than using the same action in districts nine and ten. More interestingly, there is very little overlap between the successful players who focused on districts nine and ten, which can be seen in the later half of the animation as cyan lines (with high green and blue channels). This is promising for the efficacy of the game as a computational thinking learning tool, as the players found success in various ways by committing to different long term strategies (KI.5).

Example 3. Our expert is also interested in analysis of game balance. Using the tool, we can explore individual actions, overlap them with other actions, and localize them to specific districts. Through such exploration, we identified an unexpectedly frequent use of the *voter registration drive* action among higher-scoring players (KI.3), especially in the last three levels of the game. We used additive layers encoding color and full opacity for the voter drive action in every district, along with a baseline layer adding a small amount of opacity to all lines, producing an animated pulse that highlights turns with these actions. In an attempt to isolate high voter drive usage in specific districts, we divided districts into groups of two to four (depending on the total number of districts in the level we were analyzing) and assigned groups dedicated color channels, with the plan to drill down into the groups of districts with the higher represented colors. To our surprise, even after shuffling color usage and groups, the animations always remained very noisy and colorful, due to this action being taken in most districts, and often across two districts in one turn.

The wide use of this action is not necessarily an issue, but it was designed to be more situational, and therefore deserves some analytical attention. It is also possible that this action is used by players

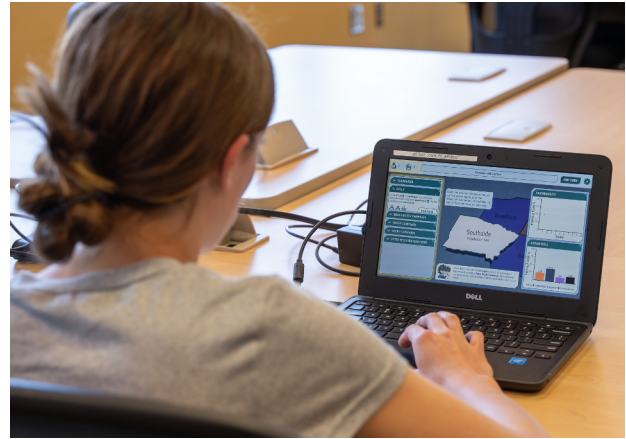


Figure 4: A GEAR UP cohort student plays Operation: ELECT, viewing one of four districts in level one. The left side of the screen shows available actions that may be taken in the selected district each turn. The right side of the screen shows district analytics the player must interpret to make informed decisions.

who have already been successful and are simply using up their extra budget. Although it does not appear to cause steep increases in overall votes when used, further analysis may be needed to determine if it is due to imbalance or simply a tool that users have successfully incorporated into their strategies.

5 CONCLUSIONS AND FUTURE WORK

We present a kinetic query visualization design and its useful application in the analysis of educational game data. Animated visualizations generated with the implemented tool demonstrate its exploratory capabilities and utility for visual analysis. Specification of kinetic queries in the tool is relatively simple and straightforward, but also suggests conveniences and extensions to enhance the effectiveness of the animated encodings. We hope to achieve greater flexibility by incorporating logical grouping of layers, fine-grain control over other visual encoding channels, and more robust temporal configurations. We also plan to include more operators and blending functions to allow for more animation effects and address issues such as overplotting. More functional operators could address one particular interest, as stated by our domain expert, to highlight phenomena in later turns based on specific conditions of earlier turns. These additions would make the technique more robust, supporting the analysis of other educational games and other domains in general.

The kinetic queries technique shows promise in revealing patterns in complicated data. It may also serve as a basis for the development of presentation and data storytelling techniques. While animation performance was sufficient in our game data application, it would likely be an issue for more complex query expressions and larger amounts of data. Moving forward, we plan to study the limitations of the technique's effectiveness and performance as we expand the expressiveness of its kinetic querying features beyond the color channels and blending modes that we used to animate the line chart in this particular application.

ACKNOWLEDGMENTS

The authors wish to thank the K20 Center at the University of Oklahoma, whose Game-Based Learning activity and data supported this research. This work was supported in part by the K20 Center's GEAR UP for LIFE grant, GEAR UP for OKC grant, GEAR UP for the FUTURE grant, and GEAR UP for MY SUCCESS grant.

REFERENCES

- [1] Advanced Distributed Learning Initiative. Experience API (xAPI) Standard, 2020. [3](#)
- [2] C. Ahlberg and B. Shneiderman. Visual information seeking: Tight coupling of dynamic query filters with starfield displays. In *Proceedings of the SIGCHI conference on Human factors in computing systems*, pp. 313–317, 1994. [1](#)
- [3] C. Ahlberg, C. Williamson, and B. Shneiderman. Dynamic queries for information exploration: An implementation and evaluation. In *Proceedings of the SIGCHI conference on Human factors in computing systems*, pp. 619–626, 1992. [1](#)
- [4] R. J. Bobrow and A. Helsing. Kinetic visualizations: A new class of tools for intelligence analysis. In *2005 Int'l Conf. Intelligence Analysis*, 2005. [2](#)
- [5] T. Camp, W. R. Adrion, B. Bizot, S. Davidson, M. Hall, S. Hambrusch, E. Walker, and S. Zweben. Generation CS: The growth of computer science. *ACM Inroads*, 8(2):44–50, 2017. doi: 10.1145/3084362 [1](#)
- [6] Computer Science Teachers Association. CSTA K-12 Computer Science Standards, revised 2017. Technical report, Computer Science Teachers Association, New York, NY, USA, 2017. [1](#)
- [7] M. Freire, Á. Serrano-Laguna, B. Manero Iglesias, I. Martínez-Ortiz, P. Moreno-Ger, and B. Fernández-Manjón. Game learning analytics: Learning analytics for serious games. In *Learning, design, and technology: An international compendium of theory, research, practice, and policy*, pp. 1–29. Springer International Publishing, 2016. doi: 10.1007/978-3-319-17727-4_21-1 [2](#)
- [8] M. L. Hilton and M. A. Honey. *Learning science through computer games and simulations*. National Academies Press, 2011. [2](#)
- [9] E. Hunsaker. Computational thinking. In A. Ottenbreit-Leftwich and R. Kimmons, eds., *The K-12 Educational Technology Handbook*. EdTech Books, 2023. [1](#)
- [10] K20 Center. Operation: ELECT, 2024. Available through the K20 Center's Game Portal. [1](#)
- [11] E. B. Lum, A. Stoppel, and K. L. Ma. Kinetic visualization: A technique for illustrating 3D shape and structure. In *IEEE Visualization, 2002. VIS 2002.*, pp. 435–442. IEEE, 2002. doi: 10.1109/VISUAL.2002.1183805 [2](#)
- [12] E. B. Lum, A. Stoppel, and K. L. Ma. Using motion to illustrate static 3D shape-kinetic visualization. *IEEE Transactions on Visualization and Computer Graphics*, 9(2):115–126, 2003. doi: 10.1109/TVCG.2003.1196000 [2](#)
- [13] B. Medler and B. Magerko. Analytics of play: Using information visualization and gameplay practices for visualizing video game data. *Parsons Journal for Information Mapping*, 2011. [2](#)
- [14] D. Moura, M. Seif el-Nasr, and C. D. Shaw. Visualizing and understanding players' behavior in video games: discovering patterns and supporting aggregation and comparison. In *ACM SIGGRAPH 2011 Game Papers*, pp. 1–6. Association for Computing Machinery, 2011. doi: 10.1145/2037692.2037695 [2](#)
- [15] J. L. Plass, B. D. Homer, and C. K. Kinzer. Foundations of Game-Based Learning. *Educational Psychologist*, 50(4):258–283, Oct. 2015. doi: 10.1080/00461520.2015.1122533 [2](#)
- [16] C. Romero and S. Ventura. Educational data mining and learning analytics: An updated survey. *Wiley interdisciplinary reviews: Data mining and knowledge discovery*, 10(3):e1355, 2020. [2](#)
- [17] H. Rosling, O. Rosling, and A. Rosling Rönnlund. Gapminder, 2005. [2](#)
- [18] J. A. Ruiperez-Valiente, M. J. Gomez, P. A. Martinez, and Y. J. Kim. Ideating and Developing a Visualization Dashboard to Support Teachers Using Educational Games in the Classroom. *IEEE Access*, 9:83467–83481, 2021. doi: 10.1109/ACCESS.2021.3086703 [2](#)
- [19] S. Tobias and J. D. Fletcher. What research has to say about designing computer games for learning. *Educational Technology*, pp. 20–29, 2007. [2](#)
- [20] S. Tobias, J. D. Fletcher, and A. P. Wind. Game-Based Learning. In J. M. Spector, M. D. Merrill, J. Elen, and M. J. Bishop, eds., *Handbook of Research on Educational Communications and Technology*, pp. 485–503. Springer New York, New York, NY, 2014. doi: 10.1007/978-1-4614-3185-5_38 [2](#)
- [21] B. Tversky, J. B. Morrison, and M. Betrancourt. Animation: Can it facilitate? *International Journal of Human-Computer Studies*, 57(4):247–262, 2002. doi: 10.1006/ijhc.2002.1017 [2](#)
- [22] S. Valentine. *The hidden power of blend modes in Adobe Photoshop*. Adobe Press, 2012. [3](#)
- [23] G. Wallner and S. Kriglstein. *An introduction to gameplay data visualization*, p. 231–250. ETC Press, Pittsburgh, PA, 2015. [2](#)
- [24] F. Yang, H. Goodell, R. Pickett, R. Bobrow, A. Baumann, A. Gee, and G. Grinstein. Data Exploration Combining Kinetic and Static Visualization Displays. In *Fourth International Conference on Coordinated & Multiple Views in Exploratory Visualization (CMV'06)*, pp. 21–30, July 2006. doi: 10.1109/CMV.2006.6 [2](#)